8

Introducing the C++ I/O System

Chapter Objectives

8.1 Some C++ I/O basics

8.2 Formatted I/O

8.3 Using **width()**, **precision()**,and **fill()**

8.4 Using I/O manipulators

8.5 Creating your own inserters

8.6 Creating extractors

ALTHOUGH you have been using C++-style I/O since the first chapter of this book, it is time to explore it more fully. Like its predecessor, C, the C++ language includes a rich I/O system that is both flexible and powerful. It is important to understand that C++ still supports the entire C I/O system. However, C++ supplies a complete set of object-oriented I/O routines. The major advantage of the C++ I/O system is that it can be overloaded relative to classes that you create. Put differently, the C++ I/O system allows you to seamlessly integrate new types that you create.

Like the C I/O system, the C++ object-oriented I/O system makes little distinction between console and file I/O. File and console I/O are really just different perspectives on the same mechanism. The examples in this chapter use console I/O, but the information presented is applicable to file I/O as well. (File I/O is examined in detail in Chapter 9.)

At the time of this writing, there are two versions of the I/O library in use: the older one that is based on the original specifications for C++ and the newer one defined by Standard C++. For the most part the two libraries appear the same to the programmer. This is because the new I/O library is, in essence, simply an updated and improved version of the old one. In fact, the vast majority of the differences between the two occur beneath the surface, in the way that the libraries are implemented-not in the way that they are used. From the programmer's perspective, the main difference is that the new I/O library contains a few additional features and defines some new data types. Thus, the new I/O library is essentially a superset of the old one. Nearly all programs originally written for the old library will compile without substantive changes when the new library is used. Since the old-style I/O library is now obsolete, this book describes only the new I/O library as defined by Standard C++. But most of the information is applicable to the old I/O library as well.

This chapter covers several aspects of C++'s I/O system, including formatted I/O, I/O manipulators, and creating your own I/O inserters and extractors. As you will see, the C++ I/O system shares many features with the C I/O system.

Review Skills Check

Before proceeding, you should be able to correctly answer the following questions and do the exercises.

1. Create a class hierarchy that stores information about airships. Start with a general base class called **airship** that stores the number of passengers and the amount of cargo (in pounds) that can be carried. Then create two derived classes called **airplane** and **balloon** from **airship**. Have **airplane** store the type of engine used (propeller or jet) and range, in miles. Have **balloon** store information about the type of gas used to lift the balloon (hydrogen or helium) and its maximum altitude (in feet). Create a short program that demonstrates this class hierarchy. (Your solution will, no doubt, differ from the answer shown in the back of this book. If it is functionally similar, count it as correct.)

2. What is **protected** used for?

3. Given the following class hierarchy, in what order are the constructor functions called? In what order are the destructor functions called?

```
#include <iostream>
using namespace std;

class A
{
public:
    A() { cout << "Constructing A\n"; }
```

```
    ~A() { cout << "Destructing A\n"; }
};

class B : public A
{
public:
    B() { cout << "Constructing B\n"; }
    ~B() { cout << "Destructing B\n"; }
};

class C : public B
{
public:
    C() { cout << "Constructing C\n"; }
    ~C() { cout << "Destructing C\n"; }
};

int main()
{
    C ob;

    return 0;
}
```

4. Given the following fragment, in what order are the constructor functions called?

```
class myclass : public A, public B, public C
{
    // ...
};
```

5. Fill in the missing constructor functions in this program:

```
#include <iostream>
using namespace std;

class base
{
    int i, j;
public:
    // need constructor
    void showij() { cout << i << ' ' << j << '\n'; }
};

class derived : public base
{
    int k;
public:
    // need constructor
    void show() { cout << k << ' '; showij(); }
};

int main()
```

```
{
    derived ob(1, 2, 3);

    ob.show();

    return 0;
}
```

6. In general, when you define a class hierarchy, you begin with the most _____ class and move to the most _____ class. (Fill in the missing words.)

8.1 SOME C++ I/O BASICS

Before we begin out examination of C++ I/O, a few general comments are in order. The C++ I/O system, like the C I/O system, operates through *streams*. Because of you C programming experience, you should already know what a stream is, but here is a summary. A stream is a logical device that either produces or consumes information. A stream is linked to a physical device by the C++ I/O system. All streams behave in the same manner, even if the actual physical devices they are linked to differ. Because all streams act the same, the I/O system presents the programmer with a consistent interface, even though it operates on devices with differing capabilities. For example, the same function that you use to write to the screen can be used to write to a disk file or to the printer.
As you know, when a C program begins execution, three predefined streams are automatically opened: **stdin**, **stdout**, and **stderr**. A similar thing happens when a C++ program starts running. When a C++ program starts running. When a C++ program begins, these four streams are automatically opened:

Stream	Meaning	Default Devices
cin	Standard input	Keyboard
cout	Standard output	Screen
cerr	Standard error	Screen
clog	Buffered version of **cerr**	Screen

As you have probably guessed, the streams **cin**, **cout**, and **cerr** correspond to C's **stdin**, **stdout**, and **stderr**. You have already been using **cin** and **cout**. The stream **clog** is simply a buffered version of **cerr**. Standard C++ also opens wide (16-bit) character versions of these streams called **wcin**, **wcout**, **wcerr**, and **wclog**, but we won't be using them in this book. The wide character streams exist to support languages, such as Chinese, that require large character sets.
By default, the standard streams are used to communicate with the console. However, in environments that support I/O redirection, these streams can be redirected to other devices.
As you learned in Chapter 1, C++ provides support for its I/O system in the header file **<iostream>**. In this file, a rather complicated set of class hierarchies is defined that supports I/O operations. The I/O classes begin with a system of *template classes.* Template classes, also called generic classes, will be discussed more fully in Chapter 11; briefly, a template class defines the form of a class without fully specifying the data upon which it will operate. Once a template class has been defined, specific instances of it can be created. As it relates to the I/O library, Standard C++ creates two specific versions of the I/O template classes: one for 8-bit characters and another for wide characters. This book will discuss only the 8-bit character classes, since they are by far the most frequently used.

The C++ I/O system is build upon two related, but different, template class hierarchies. The first is derived from the low-level I/O class called **basic_streambuf**. This class supplies the basic, low-level input and output operations and provides the underlying support for the entire C++ I/O system. Unless you are doing advanced I/O programming, you will not need to use **basic_streambuf** directly. The class hierarchy that you will most commonly be working with is derived from **basic_ios**. This is a high-level I/O class that provides formatting, error-checking, and status information related to stream I/O. **basic_ios** is used as a base for several derived classes, including **basic_istream**, **basic_ostream**, and **basic_iostream**. These classes are used to create streams capable of input, output, and input/output, respectively.
As explained earlier, the I/O library creates two specific versions of the class hierarchies just described: one for 8-bit characters and one for wide characters. The following table shows the mapping of the template class names to their 8-bit character-based versions (including some that will be used in Chapter 9):

Template Class	8-Bit Character-Based Class
basic_streambuf	streambuf
basic_ios	ios
basic_istream	istream
basic_ostream	ostream
basic_iostream	iostream
basic_fstream	fstream
basic_ifstream	ifstream
basic_ofstream	ofstream

The character-based names will be used throughout the remainder of this book, since they are the names that you will use in your programs. They are also the same names that were used by the old I/O library. This is why the old and the new I/O libraries are compatible at the source code level.
One last point: The **ios** class contains many member functions and variables that control or monitor the fundamental operation of a stream. It will be referred to frequently. Just remember that if you include **<iostream>** in you program, you will have access to this important class.

8.2 FORMATTED I/O

Until now, all examples in this book displayed information to the screen using C++'s default formats. However, it is possible to output information in a wide variety of forms. In fact, you can format data using C++'s I/O system in much the same way that you do using C's **printf()** function. Also, you can alter certain aspects of the way information is input.
Each stream has associated with it a set of format flags that control the way information is formatted. The **ios** class declares a bitmask enumeration called **fmtflags**, in which the values are defined:

adjustfield	floatfield	right	skipws
basefield	hex	scientific	unitbuf
boolalpha	internal	showbase	uppercase
dec	left	showpoint	
fixed	oct	showpos	

These values are used to set or clear the format flags and are defined within **ios**. If you are using an older, nonstandard compiler, it may not define the **fmtflags** enumeration type. In this case, the format flags will be encoded into a long integer.

When the **skipws** flag is set, leading whitespace characters (spaces, tabs, and newlines) are discarded when input is being performed on a stream. When **skipws** is cleared, whitespace characters are not discarded.
When the **left** flag is set, output is left justified. When **right** is set, output is right justified. When the **internal** flag is set, a numeric value is padded to fill a field by inserting spaces between any sign or base character. If none of these flags is set, output is right justified by default.
By default, numeric values are output in decimal. However, it is possible to change the number base. Setting the **oct** flag causes output to be displayed in octal. Setting the **hex** flag causes output to be displayed in hexadecimal. To return output to decimal, set the **dec** flag.
Setting **showbase** causes the base of numeric values to be shown. For example, if the conversion base is hexadecimal, the value 1F will be displayed as 0x1F.
By default, when scientific notation is displayed, the e is lowercase. Also, when a hexadecimal value is displayed, the x is lowercase, When **uppercase** is set, these characters are displayed uppercase. Setting **showpos** causes a leading plus sign to be displayed before positive values.
Setting **showpoint** causes a decimal point and trailing zeros to be displayed for all floating-point output-whether needed or not.
If the **scientific** flag is set, floating-point numeric values are displayed using scientific notation. When **fixed** is set, floating-point values are displayed using normal notation. When neither flag is set, the compiler chooses an appropriate method.
When **unitbuf** is set, Booleans can be input or output using the keywords **true** and **false**.
Since it is common to refer to the **oct**, **dec**, and **hex** fields, they can be collectively referred to as **basefield**. Similarly, the **left**, **right**,and **internal** fields can be referred to as **adjustfield**. Finally, the **scientific** and **fixed** fields can be referenced as **floatfield**.
To set a format flag, use the **setf()** function. This function is a member of **ios**. Its most common form is shown here:

```
fmtflags setf(fmtflags flags);
```

This function returns the previous settings of the format flags and turns on those flags specified by *flags*. (All other flags are unaffected.) For example, to turn on the **showpos** flag, you can use this statement:

```
stream.setf(ios::showpos);
```

Here *stream* is the stream that you wish to affect. Notice the use of the scope resolution operator. Remember, **showpos** is an enumerated constant within the **ios** class. Therefore, it is necessary to tell the compiler this fact by preceding **showpos** with the class name and the scope resolution operator. If you don't the constant **showpos** will simply not be recognized.
It is important to understand that **setf()** is a member function of the **ios** class and affects streams created by that class. Therefore, any call to **setf()** is done relative to a specific stream. There is no concept of calling **setf()** by itself. Put differently, there is no concept in C++ of global format status. Each stream maintains its own format status information individually.
It is possible to set more than one flag in a single call to **setf()** rather than making multiple calls. To do this, OR together the values of the flags you want to set. For example, this call sets the **showbase** and **hex** flags for **cout**:

```
cout.setf(ios::showbase | ios::hex);
```

Remember: *Because the format flags are defined within the* ***ios*** class, you must access their values by using **ios** and the scope resolution operator. For example, **showbase** by itself will not be recognized; you must specify **ios::showbase.**
The complement of **setf()** is **unsetf()**.This member function of **ios** clears one or more format flags. Its most common prototype form is shown here:

```
void unsetf(fmtflags flags);
```

The flags specified by *flags* are cleared. (All other flags are unaffected.)
There will be times when you want to know, but not alter, the current format settings. Since both **setf()** and **unsetf()** alter the setting of one or more flags, **ios** also includes the member function **flags()**, which simply returns the current setting for each format flag. Its prototype is shown here:

```
fmtflags flags();
```

The **flags()** function has a second form that allows you to set *all* format flags associated with a stream to those specified in the argument to **flags()**. The prototype for this version of **flags()** is shown here:

```
fmtflags flags(fmtflags f);
```

When you use this version, the bit pattern found in f is copied to the variable used to hold the format flags associated with the stream, thus overwriting all previous flag settings. The function returns the previous settings.

EXAMPLES

1. Here is an example that shows how to set several of the format flags:

```
#include <iostream>
using namespace std;

int main()
{
    // display using default settings
    cout << 123.23 << " hello " << 100 << '\n';
    cout << 10 << ' ' << -10 << '\n';
    cout << 100.0 << "\n\n";

    // now, change formats
    cout.unsetf(ios::dec); // not required by all compilers
    cout.setf(ios::hex | ios::scientific);
    cout << 123.23 << " hello " << 100 << '\n';

    cout.setf(ios::showpos);
    cout << 10 << ' ' << -10 << '\n';

    cout.setf(ios::showpoint | ios::fixed);
    cout << 100.0;

    return 0;
}
```

This program displays the following output:

```
123.23 hello 100
10 -10
100

1.232300e+02 hello 64
a fffffff6
```

```
+100.000000
```

Notice that the **showpos** flag affects only decimal output. It does not affect the value 10 when output in hexadecimal. Also notice the **unsetf()** call that turns off the **dec** flag (which is on by default). This call is not needed for all compilers. But for some compilers, the **dec** flag overrides the other flags, so it is necessary to turn it off when turning on either **hex** or **oct**. In general, for maximum portability, it is better to set only the number base that you want to use and clear the others.

2. The following program illustrates the effect of the **uppercase** flag. It first sets the **uppercase**, **showbase**, and **hex** flags. It then outputs 99 in hexadecimal. In this case, the X used in the hexadecimal notation is uppercase. Next, it clears the **uppercase** flag by using **unsetf()** and again outputs 88 in hexadecimal. This time, the x is lowercase.

```
#include <iostream>
using namespace std;

int main()
{
    cout.unsetf(ios::dec);
    cout.setf(ios::uppercase | ios::showbase | ios::hex);

    cout << 88 << '\n';

    cout.unsetf(ios::uppercase);

    cout << 88 << '\n';

    return 0;
}
```

3. The following program uses **flags()** to display the settings of the format flags relative to **cout**. Pay special attention to the **showflags()** function. You might find it useful in programs you write.

```
#include <iostream>
using namespace std;

void showflags();

int main()
{
    // show default condition of format flags
    showflags();

    cout.setf(ios::oct | ios::showbase | ios::fixed);

    showflags();

    return 0;
}

// This function displays the status of the format flags.
void showflags()
```

```
{
    ios::fmtflags f;

    f = cout.flags(); // get flag settings

    if(f & ios::skipws)
        cout << "skipws on\n";
    else
        cout << "skipws off\n";

    if(f & ios::left)
        cout << "left on\n";
    else
        cout << "left off\n";

    if(f & ios::right)
        cout << "right on\n";
    else
        cout << "right off\n";

    if(f & ios::internal)
        cout << "internal on\n";
    else
        cout << "internal off\n";

    if(f & ios::dec)
        cout << "dec on\n";
    else
        cout << "dec off\n";

    if(f & ios::oct)
        cout << "oct on\n";
    else
        cout << "oct off\n";

    if(f & ios::hex)
        cout << "hex on\n";
    else
        cout << "hex off\n";

    if(f & ios::showbase)
        cout << "showbase on\n";
    else
        cout << "showbase off\n";

    if(f & ios::showpoint)
        cout << "showpoint on\n";
    else
        cout << "showpoint off\n";

    if(f & ios::showpos)
```

```
        cout << "showpos on\n";
    else
        cout << "showpos off\n";

    if(f & ios::uppercase)
        cout << "uppercase on\n";
    else
        cout << "uppercase off\n";

    if(f & ios::scientific)
        cout << "scientific on\n";
    else
        cout << "scientific off\n";

    if(f & ios::fixed)
        cout << "fixed on\n";
    else
        cout << "fixed off\n";

    if(f & ios::unitbuf)
        cout << "unitbuf on\n";
    else
        cout << "unitbuf off\n";

    if(f & ios::boolalpha)
        cout << "boolalpha on\n";
    else
        cout << "boolalpha off\n";

    cout << "\n";
}
```

Inside **showflags()**, the local variable **f** is declared to be of type **fmtflags**. If your compiler does not define **fmtflags**, declare this variable as **long** instead. The output from the program is shown here:

```
skipws on
left off
right off
internal off
dec on
oct off
hex off
showbase off
showpoint off
showpos off
uppercase off
scientific off
```

```
fixed off
unitbuf off
boolalpha off

skipws on
left off
right off
internal off
dec on
oct on
hex off
showbase on
showpoint off
showpos off
uppercase off
scientific off
fixed on
unitbuf off
boolalpha off
```

4. The next program illustrates the second version of **flags()**. It first constructs a flag mask that turns on **showpos**, **showbase**, **oct**, and **right**. It then uses **flags()** to set the flag variable associated with **cout** to these settings. The function **showflags()** verifies that the flags are set as indicated. (This is the same function used in the previous program.)

```
#include <iostream>
using namespace std;

void showflags();

int main()
{
    // show default condition of format flags
    showflags();

    // showpos, showbase, oct, right are on, others off
    ios::fmtflags f = ios::showpos | ios::showbase | ios::oct
        | ios::right;

    cout.flags(f); // set flags

    showflags();

    return 0;
}
```

EXERCISES

1. Write a program that sets **cout**'s flags so that integers display a + sign when positive values are displayed. Demonstrate that you have set the format flags correctly.

2. Write a program that sets **cout**'s flags so that the decimal point is always shown when floating-point values are displayed. Also, display all floating-point values in scientific notation with an uppercase E.

3. Write a program that saves the current state of the format flags, sets **showbase** and **hex**, and displays the value 100. Then reset the flags to their previous values.

8.3 USING width(), precision(), AND fill()

In addition to the formatting flags, there are three member functions defined by **ios** that set these format parameters: the field width, the precision, and the fill character. These are **width()**, **precision()** and **fill()**, respectively.
By default, when a value is output, it occupies only as much space as the number of characters it takes to displays it. However, you can specify a minimum field width by using the **width()** function. Its prototype is shown here:

```
streamsize width(streamsize w);
```

Here *w* becomes the field width, and the previous field width is returned. The **streamsize** type is defined by **<iostream>** as some form of integer. In some implementations, each time an output operation is performed, the filed width returns to its default setting, so it might be necessary to set the minimum field width before each output statement.
After you set a minimum field width, when a value uses less than the specified width, the field is padded with the current fill character (the space, by default) so that the field width is reached. However, keep in mind that if the size of the output value exceeds the minimum field width, the field will be overrun. No values are truncated.
By default, six digits of precision are used. You can set this number by using the **precision()** function. Its prototype is shown here:

```
streamsize precision(streamsize p);
```

Here the precision is set to *p* and the old value is returned.
By default, when a field needs to be filled, it is filled with spaces. You can specify the fill character by using the **fill()** function. Its prototype is shown here:

```
char fill(char ch);
```

After a call to **fill()**, *ch* becomes the new fill character, and the old one is returned.

EXAMPLES

1. Here is a program that illustrates the format functions:

```
#include <iostream>
using namespace std;

int main()
{
    cout.width(10);           // set minimum filed width
    cout << "hello" << '\n'; // right-justify by default
    cout.fill('%');           // set fill character
```

```
    cout.width(10);           // set width
    cout << "hello" << '\n'; // right-justify default
    cout.setf(ios::left);     // left-justify
    cout.width(10);           // set width
    cout << "hello" << '\n'; // output left justified

    cout.width(10);           // set width
    cout.precision(10);       // set 10 digits of precision
    cout << 123.234567 << '\n';
    cout.width(10);           // set width
    cout.precision(6);        // set 6 digits of precision
    cout << 123.234567 << '\n';

    return 0;
}
```

This program displays the following output:

```
     hello
%%%%%hello
hello%%%%%
123.234567
123.235%%%
```

Notice that the field width is set before each output statement.

2. The following program shows how to use the C++ I/O format functions to create an aligned table of numbers:

```
// Create a table of square roots and squares.
#include <iostream>
#include <cmath>
using namespace std;

int main()
{
    double x;

    cout.precision(4);
    cout << "       x    sqrt(x)    x^2\n\n";
    for(x = 2.0; x <= 20.0; x++)
    {
        cout.width(7);
        cout << x << " ";
        cout.width(7);
        cout << sqrt(x) << " ";
        cout.width(7);
        cout << x*x << '\n';
    }

    return 0;
}
```

This program creates the following table:

```
      x   sqrt(x)   x^2


      2   1.414       4
      3   1.732       9
      4       2      16
      5   2.236      25
      6   2.449      36
      7   2.646      49
      8   2.828      64
      9       3      81
     10   3.162     100
     11   3.317     121
     12   3.464     144
     13   3.606     169
     14   3.742     196
     15   3.873     225
     16       4     256
     17   4.123     289
     18   4.243     324
     19   4.359     361
     20   4.472     400
```

EXERCISES

1. Create a program that prints the natural log and base 10 log of the numbers from 2 to 100. Format the table so that the numbers are right justified within a field width of 10, using a precision of five decimal places.

2. Create a function called **center()** that has this prototype:

   ```
   void center(char *s);
   ```

   Have this function center the specified string on the screen. To accomplish this, use the **width()** function. Assume that the screen is 80 characters wide. (For simplicity, you may assume that no string exceeds 80 characters.) Write a program that demonstrates that your function works.

3. On your own, experiment with the format flags and the format functions. Once you become familiar with the C++ I/O system, you will have no trouble using it to format output any way you like.

8.4 USING I/O MANIPULATORS

There is a second way that you can format information using C++'s I/O system. This method uses special functions called *I/O manipulators*. As you will see, I/O manipulators are, in some situations, easier to use than the **ios** format flags and functions.

I/O manipulators are special I/O format functions that can occur *within* an I/O statement, instead of separate from it the way the **ios** member functions must. The standard manipulators are shown in Table 8-1. As you can see, many of the I/O manipulators parallel member functions of the **ios** class. Many of the manipulators shown in Table 8-1 were added recently to Standard C++ and will be supported only by modern compilers.

Manipulator	**Purpose**	**Input/Output**
boolalpha	Turns on **boolalpha** flag	Input/Output
dec	Turns on **dec** flag	Input/Output
endl	Outputs a newline character and flushes the stream	Output
ends	Outputs a null	Output
fixed	Turns on **fixed** flag	Output
flush	Flushes a stream	Output
hex	Turns on **hex** flag	Input/Output
internal	Turns on **internal** flag	Output
left	Turns on **left**	Output
noboolalpha	Turns off **boolalpha** flag	Input/Output
noshowbase	Turns off **showbase** flag	Output
noshowpoint	Turns of **showpoint** flag	Output
noshowpos	Turns off **showpos** flag	Output
noskipws	Turns off **skipws** flag	Input
nounitbuf	Turns off **unitbuf** flag	Output
nouppercase	Turns off **uppercase** flag	Output
oct	Turns on **oct** flag	Input/Output
resetiosflags(fmtflags *f*)	Turns off the flags specified in f	Input/Output
right	Turns on **right** flag	Output
scientific	Turns on **scientific** flag	Output
setbase(int *base*)	Sets the number base to *base*	Input/Output
setfill(int *ch*)	Sets the fill character to *ch*	Output
setiosflags(fmtflags *f*)	Turns on the flags specified in *f*	Input/Output
setprecision(int *p*)	Sets the number of digits of precision	Output
setw(int *w*)	Sets the field width to *w*	Output
showbase	Turns on **showbase** flag	Output
showpoint	Turns on **showpoint** flag	Output
showpos	Turns on **showpos** flag	Output
skipws	Turns on **skipws** flag	Input
unitbuf	Turns on **unitbuf** flag	Output
uppercase	Turns on **uppercase** flag	Output
ws	Skips leading white space	Input

Table 8.1: The Standard C++ I/O Manipulators

To access manipulators that take parameters, such as **setw()**, you must include **<iomanip>** in you program. This is not necessary when you are using a manipulator that does not require an argument.

As stated above, the manipulators can occur in the chain of I/O operations. For example:

```
cout << oct << 100 << hex << 100;
cout << setw(10) << 100;
```

The first statement tells **cout** to display integers in octal and then outputs 100 in octal. It then tells the stream to display integers in hexadecimal and then outputs 100 in hexadecimal format. The second statement sets the field width to 10 and then displays 100 in hexadecimal format again. Notice that when a manipulator does not take an argument, such as **oct** in the example, it is not followed by parentheses. This is because it is the address of the manipulator that is passed to the overloaded << operator.

Keep in mind that an I/O manipulator affects only the stream of which the I/O expression is a part. I/O manipulators do *not* affect all streams currently opened for use.

As the preceding example suggests, the main advantages of using manipulation over the **ios** member functions is that they are often easier to use and allow more compact code to be written.

If you wish to set specific format flags manually by using a manipulator, use **setiosflags()**. This manipulator performs the same function as the member function **setf()**. To turn off flags use the **resetiosflags()** manipulator. This manipulator is equivalent to **unsetf()**.

EXAMPLES

1. This program demonstrates several of the I/O manipulators:

```
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
    cout << hex << 100 << endl;
    cout << oct << 10 << endl;

    cout << setfill('X') << setw(10);
    cout << 100 << " hi " << endl;

    return 0;
}
```

   This program displays the following:

```
64
12
XXXXXXX144 hi
```

2. Here is another version of the program that displays a table of the squares and square roots of the numbers 2 through 20. This version uses I/O manipulators instead of member functions and format flags.

```
/*
   This version uses I/O manipulators to display
   the table of squares and square roots.
*/
#include <iostream>
#include <iomanip>
#include <cmath>
```

```
using namespace std;

int main()
{
    double x;

    cout << setprecision(4);
    cout << "       x    sqrt(x)    x^2\n\n";

    for(x = 2.0; x <= 20.0; x++)
    {
        cout << setw(7) << x << " ";
        cout << setw(7) << sqrt(x) << " ";
        cout << setw(7) << x*x << '\n';
    }
}
```

3. One of the most interesting format flags added by the new I/O library is **boolalpha**. This flag can be set either directly or by using the new manipulators **boolalpha** or **noboolalpha**. What makes **boolalpha** so interesting is that setting it allows you to input and output Boolean values using the keywords **true** and **false**. Normally you must enter 1 for true and 0 for false. For example, consider the following program:

```
// Demonstrate boolalpha format flag.
#include <iostream>
using namespace std;

int main()
{
    bool b;

    cout << "Before setting boolalpha flag: ";
    b = true;
    cout << b << " ";
    b = false;
    cout << b << endl;

    cout << "After setting boolalpha flag: ";
    b = true;
    cout << boolalpha << b << " ";
    b = false;
    cout << b << endl;

    cout << "Enter a Boolean value: ";
    cin >> boolalpha >> b; // you can enter true or false
    cout << "You entered " << b;

    return 0;
}
```

Here is a sample run:

```
Before setting boolalpha flag:  1 0
```

```
After setting boolalpha flag:  true false

Enter a Boolean value:  true

You entered true
```

As you can see, once the **boolalpha** flag has been set, Boolean values are input and output using the words **true** or **false**. Notice that you must set the **boolalpha** flags for **cin** and **cout** separately. As with all format flags, setting **boolalpha** for one stream does not imply that it is also set for another.

EXERCISES

1. Redo Exercises 1 and 2 from Section 8.3, this time using I/O manipulators instead of member functions and format flags.
2. Show the I/O statement that outputs the value 100 in hexadecimal with the base indicator (the 0x) shown. Use the **setiosflags()** manipulator to accomplish this.
3. Explain the effect of setting the **boolalpha** flag.

8.5 CREATING YOUR OWN INSERTERS

As stated earlier, one of the advantages of the C++ I/O system is that you can overload the I/O operators for classes that you create. By doing so, you can seamlessly incorporate your classes into your C++ programs. In this section you learn how to overload C++'s output operator <<.

In the language of C++, the output operation is called an *insertion* and the << is called the *insertion operator*. When you overload the << for output, you are creating an *inserter function*, or *inserter* for short. The rationale for these terms comes from the fact that an output operator *inserts* information into a stream.

All inserted functions have this general form:

```
ostream &operator<<(ostream &stream, class_name obj)
{
    // body of inserter
    return stream;
}
```

The first parameter is a reference to an object of type **ostream**. This means that *stream* must be an output stream. (Remember, **ostream** is derived form the **ios** class.) The second parameter receives the object that will be output. (This can also be a reference parameter, if that is more suitable to your application.) Notice that the inserted function returns a reference to *stream*, which is of type **ostream**. This is required if the overloaded << is going to be used in a series of I/O expressions, such as

```
cout << ob1 << ob2 << ob3;
```

Within an inserter you can perform any type of procedure. What an inserted does is completely up to you. However, for the inserter to be consistent with good programming practices, you should limit its operations to outputting information to a stream.

Although you might find this surprising at first, an inserter *cannot* be a member of of the class on which it is designed to operate. Here is why: When an operator function of any type is a member of a class, the left operand, which is passed implicitly through the **this** pointer, is the object that generates the call to the operator function. This implies that the left operand is an

object of that class. Therefore, if an overloaded operator function is a member of a class the left operand must be an object of that class. However, when you create an inserter, the left operand is a stream and the right operand is the object that you want to output. Therefore, an inserter cannot be a member function.

The fact that an inserter cannot be a member function might appear to be a serious flaw in C++ because it seems to imply that all data of a class that will be output using an inserter will need to be public, thus violating the key principle of encapsulation. However, this is not the case. Even though inserters cannot be members of the class upon which they are designed to operate, they can be friends of the class. In fact, in most programming situations you will encounter, an overloaded inserter will be a friend of the class for which it was created.

EXAMPLES

1. As a simple first example, this program contains an inserter for the **coord** class, developed in a previous chapter:

```
// Use a friend inserter for objects of type coord.
#include <iostream>
using namespace std;

class coord
{
    int x, y;
public:
    coord() { x = 0; y = 0; }
    coord(int i, int j) { x = i; y = j; }
    friend ostream &operator<<(ostream &stream, coord ob);
};

ostream &operator<<(ostream &stream, coord ob)
{
    stream << ob.x << ", " << ob.y << '\n';
    return stream;
}

int main()
{
    coord a(1, 1), b(10, 23);

    cout << a << b;

    return 0;
}
```

This program displays the following:

```
1, 1
10, 23
```

The inserter in this program illustrates one very important point about creating your own inserters: make them as general as possible. In this case, the I/O statement inside the inserter outputs the values of **x** and **y** to **stream**, which is whatever stream is passed to the function. As you will see in the following chapter, when written correctly the same

inserter that outputs to the screen can be used to output to *any* stream. Sometimes beginners are tempted to write the **coord** inserter like this:

```
ostream &operator<<(ostream &stream, coord ob)
{
    cout << ob.x << ", " << ob.y << '\n';
    return stream;
}
```

In this case, the output statement is hard-coded to display information on the standard output device linked to **cout**. However, this prevents the inserter from being used by other streams. The point is that you should make your inserters as general as possible because there is no disadvantage to doing so.

2. For the sake of illustration, here is the preceding program revised so that the inserter is *not* a friend of the **coord** class. Because the inserter does not have access to the private parts of **coord**, the variables **x** and **y** have to be made public.

```
/*
   Create an inserter for objects of type coord, using
   a non-friend inserter
*/
#include <iostream>
using namespace std;

class coord
{
public:
    int x, y; // must be public
    coord() { x = 0; y = 0; }
    coord(int i, int j) { x = i; y = j; }
};

// An inserter for the coord class.
ostream &operator<<(ostream &stream, coord ob)
{
    stream << ob.x << ", " << ob.y << '\n';
    return stream;
}

int main()
{
    coord a(1, 1), b(10, 23);

    cout << a << b;

    return 0;
}
```

3. An inserter is not limited to displaying only textual information. An inserter can perform any operation or conversion necessary to output information in a form needed by a particular device or situation. For example, it is perfectly valid to create an inserter that sends information to a plotter. In this case, the inserted will need to send appropriate

plotter codes in addition to the information. To allow you to taste the flavor of this type of inserter, the following program creates a class called **triangle**, which stores the width and height of a right triangle. The inserter for this class displays the triangle on the screen.

```
// This program draws right triangles
#include <iostream>
using namespace std;

class triangle
{
    int height, base;
public:
    triangle(int h, int b) { height = h; base = b; }
    friend ostream &operator<<(ostream &stream, triangle ob);
};

// Draw a triangle
ostream &operator<<(ostream &stream, triangle ob)
{
    int i, j, h, k;

    i = j = ob.base-1;
    for(h=ob.height-1; h; h--)
    {
        for(k=i; k; k--)
            stream << ' ';
        stream << '*';

        if(j!=i)
        {
            for(k=j-i-1; k; k--)
                stream << ' ';
            stream << '*';
        }

        i--;
        stream << '\n';
    }
    for(k=0; k<ob.base; k++)
        stream << '*';
    stream << '\n';

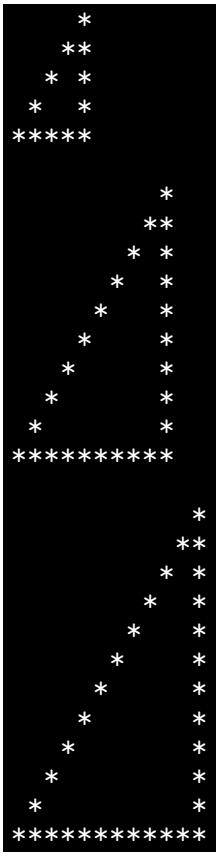
    return stream;
}

int main()
{
    triangle t1(5, 5), t2(10, 10), t3(12, 12);

    cout << t1;
    cout << endl << t2 << endl << t3;
```

```
    return 0;
}
```

Notice that this program illustrates how a properly designed inserter can be fully integrated into a "normal" I/O expression. This program displays the following:



EXERCISES

1. Given the following **strtype** class and partial program, create an inserter that displays a string:

```
#include <iostream>
#include <cstring>
#include <cstdlib>
using namespace std;

class strtype
{
    char *p;
    int len;
public:
    strtype(char *ptr);
    ~strtype() { delete [] p; }
    friend ostream &operator<<(ostream &stream, strtype &ob);
};

strtype::strtype(char *ptr)
{
```

```
    len = strlen(ptr)+1;
    p = new char (len);
    if(!p)
    {
        cout << "Allocation error\n";
        exit(1);
    }
    strcpy(p, ptr);
}

// Create operator<< inserter function here.

int main()
{
    strtype s1("This is a test."), s2("I like C++.");

    cout << s1 << '\n' << s2;

    return 0;
}
```

2. Replace the **show()** function in the following program with an inserter function:

```
#include <iostream>
using namespace std;

class planet
{
protected:
    double distance; // miles from the sun
    int revolve; // in days
public:
    planet(double d, int r)
    {
        distance = d;
        revolve = r;
    }
};

class earth : public planet
{
    double circumference; // circumference of orbit
public:
    earth(double d, int r) : planet(d, r)
    {
        circumference = 2*distance*3.1416;
    }
    /*
       Rewrite this so that it displays the information using
       an inserter function.
    */
    void show()
```

```
    {
        cout << "Distance from sum: " << distance << '\n';
        cout << "Days in orbit: " << revolve << '\n';
        cout << "Circumference of orbit: " << circumference
           << '\n';
    }
};

int main()
{
    earth ob(93000000, 365);

    cout << ob;

    return 0;
}
```

3. Explain why an inserter cannot be a member function.

8.6 CREATING EXTRACTORS

Just as you can overload the << output operator, you can overload the >> input operator. In C++, the >> is referred to as the *extractor*. The reason for this term is that the act of inputting information from a stream removes (that is, extracts) data from it.
The general form of an extractor function is shown here:

```
istream &operator>>(istream &stream, class_name &ob)
{
    // body of extractor
    return stream;
}
```

Extractors return a reference to **istream**, which is an input stream. The first parameter must be a reference to an input stream. The second parameter is a reference to the object that is receiving input.
For the same reason that an inserter cannot be a member function, an extractor cannot be a member function. Although, you can perform any operation within an extractor, it is best to limit its activity to inputting information.

EXAMPLES

1. This program adds an extractor to the **coord** class:

```
// Use a friend extractor for objects of type coord.
#include <iostream>
using namespace std;

class coord
{
    int x, y;
public:
    coord() { x = 0; y = 0; }
    coord(int i, int j) { x = i; y = j; }
```

```
    friend ostream &operator<<(ostream &stream, coord ob);
    friend istream &operator>>(istream &stream, coord &ob);
};

ostream &operator<<(ostream &stream, coord ob)
{
    stream << ob.x << ", " << ob.y << '\n';
    return stream;
}

istream &operator>>(istream &stream, coord &ob)
{
    cout << "Enter coordinates: ";
    stream >> ob.x >> ob.y;
    return stream;
}

int main()
{
    coord a(1, 1), b(10, 23);

    cout << a << b;

    cin >> a;
    cout << a;

    return 0;
}
```

Notice how the extractor also prompts the user for input. Although such prompting is not required (or even desired) for most situations, this function shows how a customized extractor can simplify coding when a prompting message is needed.

2. Here an inventory class is created that stores the name of an item, the number on hand, and its cost. The program includes both an inserter and an extractor for this class.

```
#include <iostream>
#include <cstring>
using namespace std;

class inventory
{
    char item[40]; // name of item
    int onhand; // number on hand
    double cost; // cost of item
public:
    inventory(char *i, int o, double c)
    {
        strcpy(item, i);
        onhand = o;
        cost = c;
    }
```

```
    friend ostream &operator<<(ostream &stream, inventory ob)
        ;
    friend istream &operator>>(istream &stream, inventory &ob
        );
};

ostream &operator<<(ostream &stream, inventory ob)
{
    stream << ob.item << ": " << ob.onhand;
    stream << " on hand at $" << ob.cost << '\n';
}

istream &operator>>(istream &stream, inventory &ob)
{
    cout << "Enter item name: ";
    stream >> ob.item;
    cout << "Enter number on hand: ";
    stream >> ob.onhand;
    cout << "Enter cost: ";
    stream >> ob.cost;

    return stream;
}

int main()
{
    inventory ob("hammer", 4, 12.55);

    cout << ob;

    cin >> ob;

    cout << ob;

    return 0;
}
```

EXERCISES

1. Add an extractor to the **strtype** class from Exercise 1 in the preceding section.
2. Create a class that stores an integer value and its lowest factor. Create both an inserter and an extractor for this class.

SKILLS CHECK

Mastery Skills Check

At this point you should be able to perform the following exercises and answer the questions.

1. Write a program that displays the number 100 in decimal, hexadecimal, and octal. (Use the **ios** format flags.)

2. Write a program that displays the value 1000.5364 in a 20-character field, left justified, with two decimal places, using * as a fill character. (Use the **ios** format flags.)

3. Rewrite your answer to Exercises 1 and 2 so that they use I/O manipulators.

4. Show how to save the format flags for **cout** and how to restore them. Use either member functions or manipulators.

5. Create an inserter and an extractor for this class:

```
class pwr
{
    int base;
    int exponent;
    double result; // base to the exponent power
public:
    pwr(int b, int e);
};

pwr::pwr(int b, int e)
{
    base = b;
    exponent = e;

    result = 1;
    for( ; e; e--)
        result = result * base;
}
```

6. Create a class called **box** that stores the dimensions of a square. Create an inserter that displays a square box on the screen. (Use any method you like to display the box.)

Cumulative Skills Check

This section checks how well you have integrated material in this chapter with that from the preceding chapters.

1. Using the **stack** class shown here, create an inserter that displays the contents of the stack. Demonstrate that your inserter works.

```
#include <iostream>
using namespace std;

#define SIZE 10

// Declare a stack class for characters
class stack
{
    char stck[SIZE]; // holds the stack
    int tos; // index of top-of-stack
public:
    stack();
    void push(char ch); // push character on stack
    char pop(); // pop character from stack
```

```
};

// Initialize the stack
stack::stack()
{
    tos = 0;
}

// Push a character
void stack::push(char ch)
{
    if(tos==SIZE)
    {
        cout << "Stack is full\n";
        return 0;
    }
    stck[tos] = ch;
    tos++;
}

// Pop a character.
char stack::pop()
{
    if(tos==0)
    {
        cout << "Stack is empty\n";
        return 0; // return null on empty stack
    }
    tos--;
    return stck[tos];
}
```

2. Write a program that contains a class called **watch**. Using the standard time functions, have this class's constructor read the system time and store it. Create an inserter that displays the time.

3. Using the following class, which converts feet to inches, create an extractor that prompts the user for feet. Also, create an inserter that displays the number of feet and inches. Include a program that demonstrates that your inserter and extractor work.

```
class ft_to_inches
{
    double feet;
    double inches;
public:
    void set(double f)
    {
        feet = f;
        inches = f * 12;
    }
};
```